National Institute of Technology, Warangal
Department of Computer Science and Engineering

Assignment 6

Call Graph, Module Structure & Coupling Analysis

Monte Carlo Based Risk Analysis Engine

with Backtesting Framework

Name Abhinav Singh Basnet
Roll No 24CSB0B88

Name Nithik Deva
Roll No 24CSB0B87

Course Software Engineering (CS1204)
Instructor Dr. Sangharatna Godboley
Submitted February 26, 2026

Contents

1 Source Code **2**

1.1 Entry Point and Orchestration — `main()` 2

1.2 Data Ingestion — `load_data()` and `validate_data()` 2

1.3 Rolling Window Generation — `generate_rolling_windows()` 4

1.4 Monte Carlo Simulation — `run_monte_carlo()` 4

1.5 Risk Metrics — `calculate_var()` and `calculate_mean_return()` 5

1.6 Optimisation Core — `find_optimal_window()` 5

1.7 Output and Logging — `generate_report()` and `log()` 6

2 Call Graph **7**

2.1 Call Graph Diagram 8

2.2 Explanation of the Diagram 8

3 Module Structure **9**

3.1 Module Structure Diagram 9

3.2 Explanation of Module Structure 9

4 Coupling and Cohesion Analysis **10**

4.1 Reverse Engineering the Source Code 10

4.2 Coupling Diagram 11

4.3 Types of Coupling 12

4.4 Cohesion Assessment 12

1. Source Code

The system is implemented in Python 3.10+ using `NumPy`, `Pandas`, and `SciPy`, structured across seven logical modules.

1.1 Entry Point and Orchestration — `main()`

`main()` is the top-level controller. It contains no business logic — it only sequences the three pipeline stages and delegates to the modules below.

```python
# ============================================================
# MAIN MODULE  (Entry Point)
# ============================================================
def main():
    # Top-level controller: sequences pipeline, no business logic
    print("[INFO] Starting Monte Carlo Rolling-Window Optimiser")

    data = load_data("market_data.csv")        # Stage 1: ingest
    if not validate_data(data):                # Stage 2: validate
        print("[ERROR] Data validation failed. Aborting.")
        return

    best_window, results = find_optimal_window(data, WINDOW_SIZES)
    generate_report(best_window, results)      # Stage 3: report

if __name__ == "__main__":
    main()
```

Listing 1: Entry point — orchestrates the full pipeline

1.2 Data Ingestion — `load_data()` and `validate_data()`

`load_data()` isolates all file I/O, returning a clean `DataFrame`. `validate_data()` acts as a *guard clause*, aborting the pipeline early on bad data.

```python
# ============================================================
# DATA LOADER MODULE  (Logical Module 2)
# ============================================================
import os, pandas as pd, numpy as np
```

```python

def load_data(filepath: str) -> pd.DataFrame:
    # Isolates all file I/O; only function that touches disk
    if not os.path.exists(filepath):
        print(f"[ERROR] File not found: {filepath}")
        return None
    try:
        df = pd.read_csv(filepath)
    except Exception as e:
        print(f"[ERROR] Could not parse CSV: {e}")
        return None

    if RETURNS_COL not in df.columns:
        print(f"[ERROR] Required column '{RETURNS_COL}' missing.")
        return None

    df = df[[RETURNS_COL]].dropna().reset_index(drop=True)
    df[RETURNS_COL] = pd.to_numeric(df[RETURNS_COL], errors='coerce')
    df = df.dropna().reset_index(drop=True)
    print(f"[INFO] Loaded {len(df)} return observations.")
    return df
```

Listing 2: `load_data()` — CSV ingestion with schema check

```python
def validate_data(data: pd.DataFrame) -> bool:
    # Guard clause: any failure aborts the pipeline early
    if data is None or data.empty:
        print("[ERROR] DataFrame is empty or None."); return False

    if not pd.api.types.is_numeric_dtype(data[RETURNS_COL]):
        print("[ERROR] Returns column must be numeric."); return False

    if data[RETURNS_COL].isnull().any():
        print("[ERROR] NaN values remain in returns."); return False

    min_required = max(WINDOW_SIZES)
    if len(data) < min_required:
        print(f"[ERROR] Need >= {min_required} rows, got {len(data)}.")
        return False

```

```python
    print(f"[INFO] Validation passed ({len(data)} rows).")
    return True
```

Listing 3: `validate_data()` — pre-flight integrity checks (guard clause)

1.3 Rolling Window Generation — `generate_rolling_windows()`

Produces overlapping `DataFrame` slices using a sliding index, enabling dynamic recalibration of the Monte Carlo engine per window.

```python
# ==============================================================
# WINDOW GENERATOR MODULE  (Logical Module 3)
# ==============================================================
def generate_rolling_windows(data: pd.DataFrame,
                             window_size: int) -> list:
    # Produces (len(data) - window_size) overlapping windows
    # Each window is a DataFrame slice of exactly window_size rows
    n = len(data)
    if n < window_size:
        print(f"[WARN] window_size={window_size} > data length={n}.")
        return []
    return [
        data.iloc[i : i + window_size].reset_index(drop=True)
        for i in range(n - window_size)
    ]
```

Listing 4: `generate_rolling_windows()` — overlapping window slicer

1.4 Monte Carlo Simulation — `run_monte_carlo()`

Bootstraps `SIMULATION_RUNS` simulated returns from the empirical window distribution in a single vectorised `np.random.choice` call — no Python loop.

```python
# ==============================================================
# SIMULATION ENGINE MODULE  (Logical Module 4)
# ==============================================================
def run_monte_carlo(window_data: pd.DataFrame) -> np.ndarray:
    # Bootstrap-resample the empirical return distribution
    # np.random.choice (with replacement) runs in C -- no Python loop
    returns = window_data[RETURNS_COL].to_numpy()
    if len(returns) == 0:
        return np.array([])
    return np.random.choice(returns, size=SIMULATION_RUNS, replace=True)
```

Listing 5: `run_monte_carlo()` — vectorised bootstrap sampler

1.5 Risk Metrics — `calculate_var()` and `calculate_mean_return()`

Both are **pure functions** — stateless, side-effect-free, and independently testable. `calculate_var()` extracts the $(1-\alpha)$-th percentile as the VaR estimate; `calculate_mean_return()` is an optional diagnostic.

```python
# ============================================================
# RISK METRICS MODULE  (Logical Module 5)
# ============================================================
def calculate_var(simulations: np.ndarray,
                  confidence_level: float) -> float:
    # Historical-simulation VaR: loss exceeded with prob (1-alpha)
    # Pure function -- no side effects, referentially transparent
    if len(simulations) == 0:
        return float('nan')
    percentile = (1.0 - confidence_level) * 100.0
    return float(np.percentile(simulations, percentile))
```

Listing 6: `calculate_var()` — Value-at-Risk via percentile method

```python
def calculate_mean_return(simulations: np.ndarray) -> float:
    # Optional diagnostic -- not on the critical execution path
    # Pure function: identical inputs always produce identical output
    if len(simulations) == 0:
        return float('nan')
    return float(np.mean(simulations))
```

Listing 7: `calculate_mean_return()` — expected return diagnostic

1.6 Optimisation Core — `find_optimal_window()`

The most complex function in the system (**highest cyclomatic complexity**). Drives the nested loop over window sizes and individual windows, aggregates VaR values, and returns the `argmin`.

```python
# ============================================================
# OPTIMIZER MODULE  (Logical Module 6)
# ============================================================
def find_optimal_window(data: pd.DataFrame,
                        window_sizes: list) -> tuple:
```

```python
    # Outer loop: candidate window sizes
    # Inner loop: each rolling window within that size
    # Complexity: O(N * S) where N = windows per size, S = |window_sizes|
    results = {}
    for size in window_sizes:
        windows   = generate_rolling_windows(data, size)
        if not windows:
            continue
        var_list  = [calculate_var(run_monte_carlo(w), CONFIDENCE_LEVEL)
                     for w in windows]
        var_list  = [v for v in var_list if not np.isnan(v)]
        if not var_list:
            continue
        avg_var   = float(np.mean(var_list))
        results[size] = avg_var
        print(f"[INFO]  Window {size:>4d} days -> Avg VaR: {avg_var:.6f}")

    if not results:
        raise RuntimeError("No valid results. Check input data.")
    best_window = min(results, key=results.get)
    return best_window, results
```

Listing 8: `find_optimal_window()` — window-size search (highest cyclomatic complexity)

1.7 Output and Logging — `generate_report()` and `log()`

`generate_report()` is the terminal node — no return value, pure output. `log()` is a cross-cutting utility called by all modules.

```python
# ==============================================================
# REPORT GENERATOR MODULE  (Logical Module 7)
# ==============================================================
def generate_report(best_window: int, results: dict) -> None:
    # Terminal node -- no return value, pure output side effect
    print("\n" + "=" * 46)
    print("   Monte Carlo Window Optimisation Results")
    print("=" * 46)
    print(f"  Optimal Window  : {best_window} days")
    print(f"  Confidence Level: {CONFIDENCE_LEVEL * 100:.0f}%")
    print(f"  Simulation Runs : {SIMULATION_RUNS:,}")
    print("-" * 46)
```

```python
    for w, v in sorted(results.items()):
        tag = "  <-- optimal" if w == best_window else ""
        print(f"  Window {w:>4d} days  Avg VaR: {v:>10.6f}{tag}")
    print("=" * 46)
```

Listing 9: `generate_report()` — formatted results output (terminal node)

```python
# ============================================================
# CONFIG MODULE  (Logical Module 1)
# ============================================================
import datetime

RETURNS_COL     = "returns"   # Named constant eliminates stamp-coupling risk
WINDOW_SIZES    = [60, 90, 120, 180]
SIMULATION_RUNS = 1_000
CONFIDENCE_LEVEL = 0.95

def log(message: str) -> None:
    # Cross-cutting utility -- called by ALL modules (common coupling)
    # Centralises timestamped output; in-degree == total function count
    ts = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{ts}]  {message}")
```

Listing 10: Configuration constants and `log()` cross-cutting utility

2. Call Graph

A **call graph** $G = (V, E)$ models caller–callee relationships between functions. It is a key artefact in *static program analysis* for reasoning about control flow and module dependencies. Solid edges are required calls; dashed edges are optional or utility calls.

2.1 Call Graph Diagram

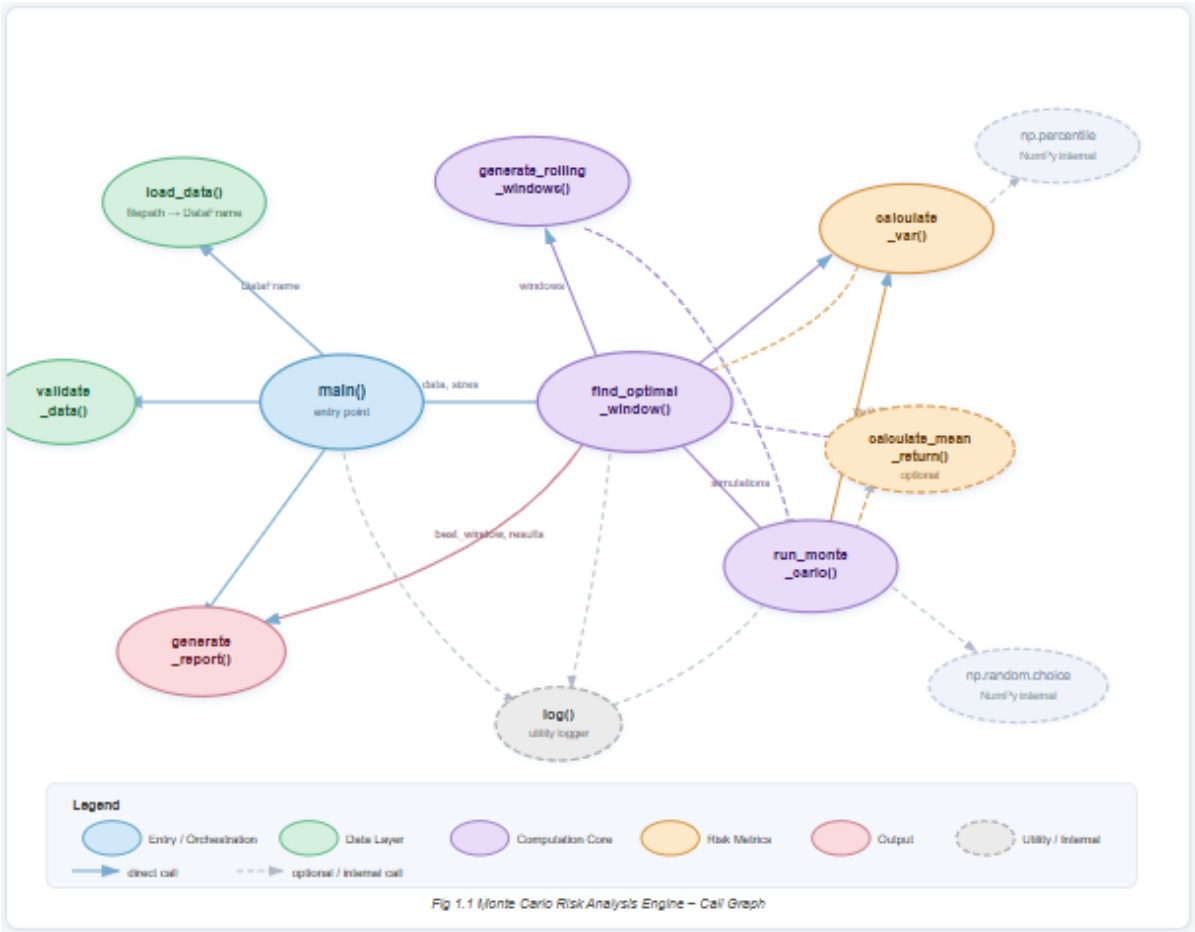

Figure 1: Call Graph of the Monte Carlo Risk Analysis Engine

2.2 Explanation of the Diagram

The call graph is structured around two **hub nodes** with high *out-degree*: `main()` (fan-out = 4) and `find_optimal_window()` (fan-out = 3 inside a loop). All leaf nodes are either pure computations or output functions with fan-out = 0.

1. Entry Point and Data Layer

`main()` is the single entry vertex, calling four functions in strict sequential order with no branching — making the graph *acyclic* at the top level. `load_data()` and `validate_data()` form a **guard clause** pattern: validation gates the pipeline before any expensive computation begins.

2. Computation Core

`find_optimal_window()` has the highest fan-out and cyclomatic complexity, calling `generate_rolling_windows()`, `run_monte_carlo()`, and `calculate_var()` inside a nested loop. It aggregates per-window VaR values and returns the `argmin`.

3. Risk Metrics and Utility

`calculate_var()` and `calculate_mean_return()` are **pure functions** — no side effects, trivially unit-testable. `log()` is a **cross-cutting concern** with dashed edges from every node; its in-degree equals the total function count.

> **Graph property summary:** The call graph is a *directed acyclic graph* (DAG) at the top level. There are no recursive calls and no cycles, which means the system has a well-defined execution order and a finite, predictable call depth. The maximum call depth is 4 (`main` → `find_optimal_window` → `run_monte_carlo` → `calculate_var`).

3. Module Structure

3.1 Module Structure Diagram

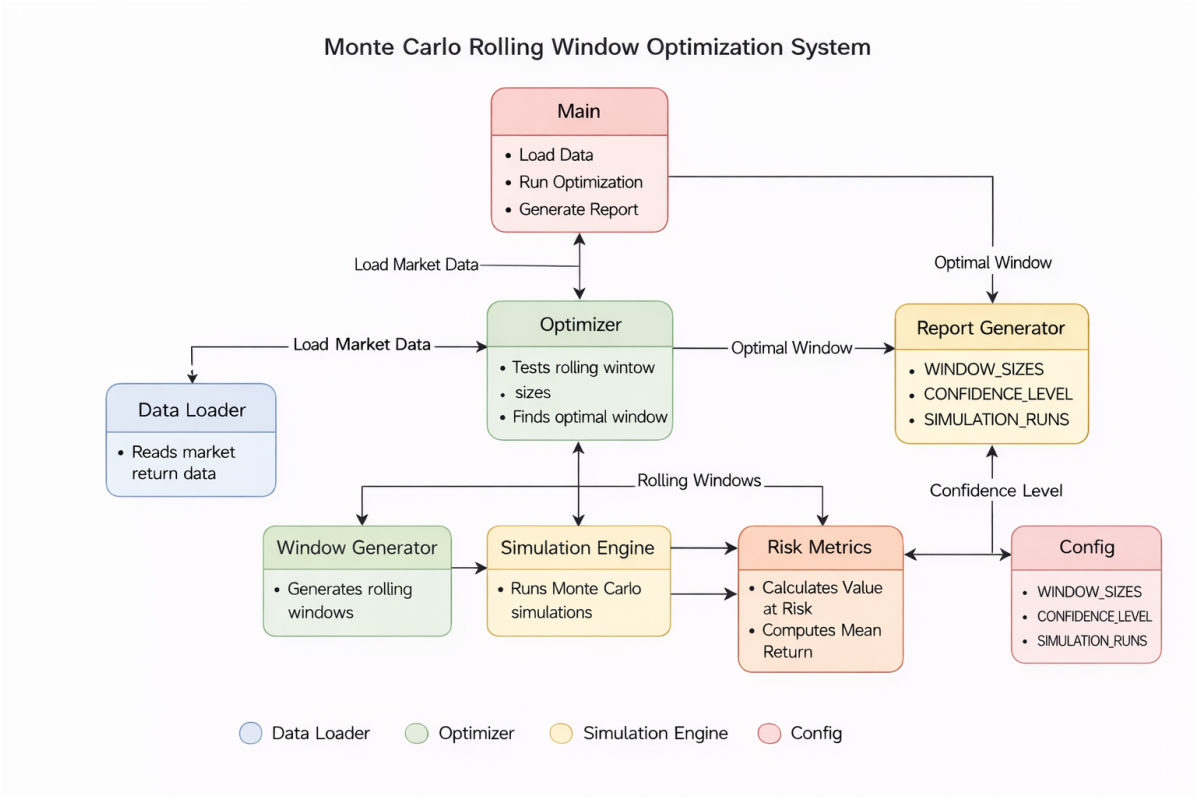

Figure 2: Hierarchical Module Structure of the Risk Analysis Engine

3.2 Explanation of Module Structure

The diagram shows the **Main** module at the top as the entry point, branching downward into six logical modules. The layout reflects a strict top-down dependency hierarchy with no upward dependencies.

Main orchestrates three operations — load data, run optimisation, generate report —

and communicates with the **Optimizer** via a bidirectional *Load Market Data* edge and with the **Report Generator** via an *Optimal Window* edge. The **Data Loader** (blue) feeds market return data leftward into the Optimizer. The **Optimizer** (green) drives the **Window Generator** and **Simulation Engine**, which feed rolling windows and simulation results into the **Risk Metrics** module. The **Config** module (pink) sits independently on the right, supplying `WINDOW_SIZES`, `CONFIDENCE_LEVEL`, and `SIMULATION_RUNS` to both the Report Generator and Risk Metrics — representing *control coupling* via shared constants.

4. Coupling and Cohesion Analysis

4.1 Reverse Engineering the Source Code

Reverse engineering the source code reveals three structural properties:

1. **No shared mutable state.** All communication is via function parameters and return values. Constants (`SIMULATION_RUNS`, `CONFIDENCE_LEVEL`) are read-only.
2. **No circular dependencies.** Module dependencies are strictly hierarchical: Reporting → Optimisation Core → Simulation / Risk Metrics → Data Ingestion.
3. **Implicit structural contract.** The column name `'returns'` is assumed across multiple modules — a mild stamp-coupling risk mitigated by declaring `RETURNS_COL = "returns"` as a named constant.

4.2 Coupling Diagram

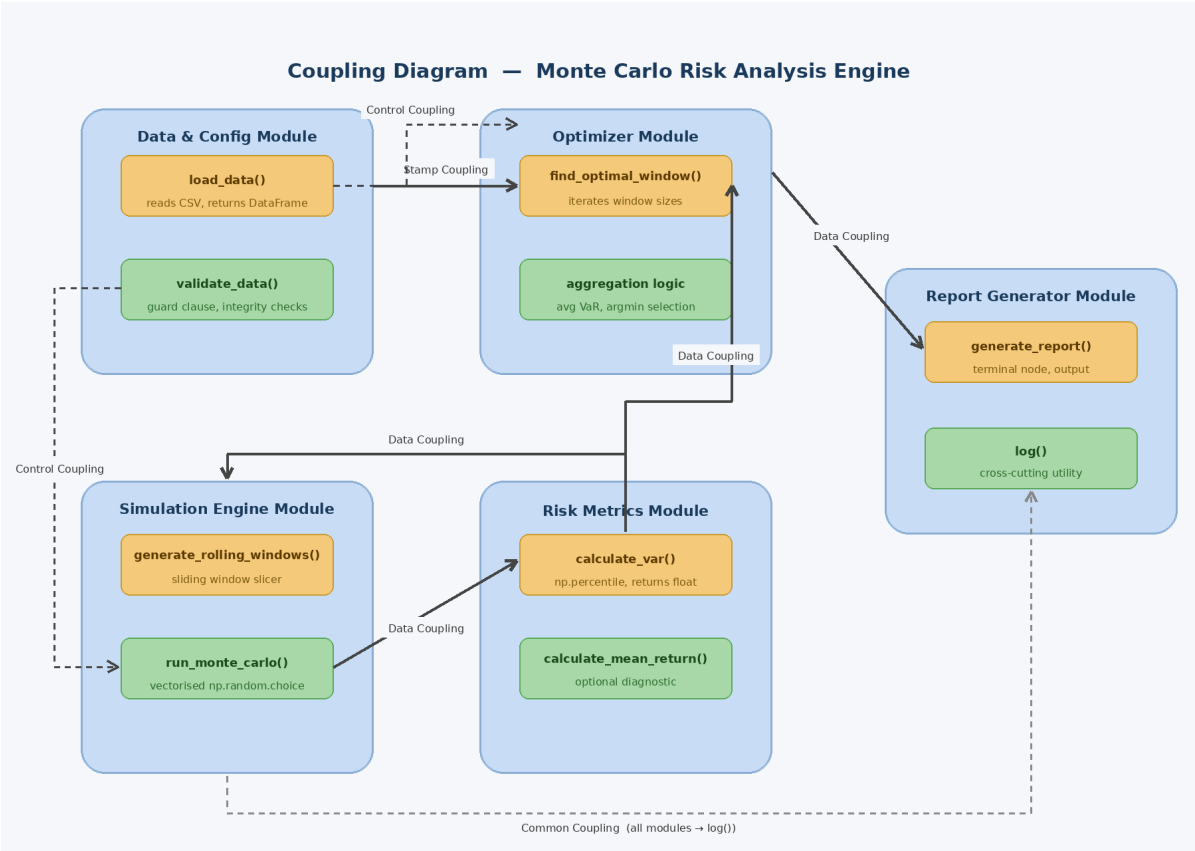

Figure 3: Coupling Diagram of the Monte Carlo Risk Analysis Engine

The diagram groups functions into five coloured module containers. Solid arrows represent **data coupling** — only primitive values or arrays are exchanged. The dashed arrow from the Data & Config module to the Optimizer represents **stamp coupling**, as a full `DataFrame` object is passed. Two dashed lines on the left edge represent **control coupling** from the Config constants into the Simulation Engine and Optimizer. The bottom dashed arc connecting all modules to `log()` in the Report Generator represents **common coupling** — the one shared resource in the system. Within each module, orange nodes are core functions and green nodes are supporting sub-functions.

4.3 Types of Coupling

Coupling Type	Modules Involved	Description
Data Coupling	`main()` → `find_optimal_window()`	A `DataFrame` and `List[int]` are passed as arguments; a `(int, Dict)` tuple is returned. No internals exposed. **Lowest and most desirable** form.
Data Coupling	`find_optimal_window()` → `run_monte_carlo()`	A `DataFrame` window slice passed in; `List[float]` simulations returned. Neither module knows the other's internals.
Data Coupling	`run_monte_carlo()` → `calculate_var()`	A flat array of floats passed in; a scalar `float` returned. Minimal, clean interface — textbook data coupling.
Stamp Coupling	`load_data()` → `validate_data()` `validate_data()` → `pipeline`	A full `DataFrame` object (composite structure) is passed and both receiver functions access the `'returns'` column by name. Mild but manageable.
Common Coupling	`All modules` → `log()`	All functions share the `log()` utility and the same output stream. Intentional and minimal; acceptable for a cross-cutting concern like logging.

Table 1: Coupling types identified in the Risk Analysis Engine

4.4 Cohesion Assessment

Each module in the system exhibits **functional cohesion** — the highest level on the Yourdon–Constantine cohesion scale — where every element within a module contributes to a single, well-defined task:

- **Simulation Engine:** handles exactly one task — generating simulated return paths. No knowledge of VaR, I/O, or reporting.
- **Risk Metrics:** pure, *referentially transparent* transformations. Identical inputs always produce identical outputs.
- **Data Ingestion:** the only module that touches the filesystem. The rest of the pipeline only ever sees a validated `DataFrame`.